

Parrales Bringas Jonathan Junior

- 1.- El Repository es una fachada porque esconde de donde salen los datos: El View Model solo lo pide como los productos. Sin saber si viene de Room o de la API por Retrofit, y por eso la UI nunca toca directamente los DAOs. Y es la única fuente de verdad porque la app lee su estado de un solo lugar autoritativo, que es Room; Así la red no escribe en pantalla, escribe en Room, y Room alimenta la pantalla evitando inconsistencias. Para coordinarlos se usa Cache then- Network: El Repository ante primero el caché de de Room al instante y a la vez pide datos frescos a la API; cuando llegan no van directo a la pantalla, sino que se guardan en Room con un Upset, y como DAO devuelve un Flow de Room, detecta el cambio y re compone la UI sola. ~~de nuevo~~
- 2.- El View Model Core es un scope de corrutinas atado al ciclo de vida del View Model. Es clave para la memoria porque, gracias a la Structured Concurrency, si el usuario cierra la pantalla antes de que llegue la respuesta de red, esa petición se cancela automáticamente al destruirse el View Model, evitando memory leaks y trabajo huérfano.
- 3.- Un flow es frío: No tiene nada hasta que un collector lo activa, y arranca de cero por cada suscriptor. Un state flow es caliente: Siempre retiene el último valor y lo tiene qpon es alguien lo observa. Para la UI se prefiere StateFlow, porque siempre tiene un estado actual disponible y comparte ese valor entre observadores, ideal para la recomposición en Compose.
- 4.- La DI Manual consiste en crear las dependencias (base de datos, repositorios) como propiedades en la clase Application, que vive durante toda la app, y de ahí inyectarlas a quien las necesite sin usar librerías como Hilt. El by lazy retrasa la creación de cada objeto hasta el primer uso evitando inicializar la B.D o el repository innecesariamente al arrancar y garantizando una sola instancia.

II)

- 1- El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo `Flow <List <T>>`
- 2- El operador `Dispatchers.Default` permite ejecutar tareas pesadas fuera del `Main thread`, < específicamente optimizado para tareas de CPU como el parseo masivo de datos.
- ~~2.1 El operador `Dispatchers.Default` permite ejecutar tareas pesadas fuera del `Main thread`, < específicamente optimizado para tareas de CPU como el parseo masivo de datos.~~
- 3- Para que un servicio en primer plano de ubicación sea válido en Android 10+, debe declararse el manifest con el tipo `foregroundServiceType = "location"`
- 4- La función `applicationContext` es de la clase `Application` es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast".
- 5- El componente que permite transformar una API basada en `Callbacks` antiguos a una función suspendible moderna se llama `CallbackFlow`
- 6- El archivo de metadatos que describe la escritura de la app al sistema operativo se llama exactamente `AndroidManifest.xml`
- 7- En la arquitectura AOSP, la capa que actúa como puente estándar entre el `Framework` y los controladores se denomina `HAL`
- 8- El operador de `Flow` que permite combinar múltiples fuentes (como `GPS` y `Sensores`) para emitir un estado unificado es `combine()`

III)

```
1. - class UsuarioRepository(private val usuarioDao: UsuarioDao) {  
    fun getUsuarios(): Flow<List<Usuario>> = usuarioDao.getAll()  
}
```

```
class MyApplication : Application() {
```

```
    val database by lazy { AppDatabase.getInstance(this) }  
    val usuarioRepository by lazy { UsuarioRepository(database.usuarioDao()) }  
}
```

2.-

// a) Data class

```
data class Usuario(  
    val nombre: String,  
    val apellidos: String,  
    val edad: Int?  
)
```

```
fun main() {
```

// b) objeto con datos reales

```
val usuario = Usuario("Jonathan", "Parrales", 30)
```

// c) puntaje de identidad = letras del nombre + letras de los apellidos

```
val apellidos puntaje = usuario.nombre?.length?.plus(usuario.apellidos.length)?.or(0)
```

// d) impresión ~~del~~ del mensaje

```
println("Usuario: $usuario.nombre | $usuario.apellidos |  
Tu puntaje de identidad es: $puntaje")  
{
```


①

1)

nivel

menor

↓

1. Aplicaciones
Apps de usuario y del sistema (API pública)

2. Framework de Android (Java API)
Clases e interfaces precompiladas para apps

3. Bibliotecas nativas + Android Runtime (ART)
libc, libbinder; ART convierte bytecode a CPU

4. HAL (Hardware Abstraction Layer)
puente framework ↔ controladores de hardware

5. Kernel de Linux
núcleo: drivers, memoria, procesos, hardware

1.- Aplicaciones: - apps terceros y del sistema, construidas con la API pública.

2.- Framework de Android: - Código Java pre compilado (clases e interfaces) que usan las apps; aquí viven componentes como el System-server.

3.- Bibliotecas nativas + Android Runtime: - librerías en C/C++ (libc, libbinder) y el ART, que traduce el bytecode de la app a instrucciones del procesador.

4.- HAL: - Capa puente estándar entre el framework y los controladores de Hardware

5.- Kernel de Linux: - El núcleo del sistema, que se comunica directamente con el hardware (drivers, memoria, procesos).